

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Rathish Madharaju	Reg.No.:	192424071
Department:	AIDS	Date:	

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu s$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. I/O Communication Mechanism for a Real-Time Embedded System

Constraints: CPU utilization < 15% · sensors interrupt every 1 ms · vectored interrupt handling required · choose memory-mapped vs I/O-mapped I/O for optimal latency.

Design

- Interrupt strategy: Use a Vectored Interrupt Controller (VIC). Each sensor line is wired to a dedicated interrupt vector, so the CPU jumps straight to the correct ISR address without polling a status register to identify the source. This removes the software overhead of a priority-encoding "interrupt identification" loop, which is essential when interrupts arrive every 1 ms.
- I/O addressing choice: Memory-mapped I/O is selected over I/O-mapped (isolated) I/O. Device registers are mapped into the normal address space, so the CPU can use ordinary load/store instructions (instead of special IN/OUT instructions) to read sensor data. This shortens the ISR instruction path length, which directly reduces CPU time spent per interrupt and helps meet the < 15% utilization budget.
- Meeting the 15% CPU budget: With 1 ms inter-arrival time, the ISR (including vector fetch, register save, data read, register restore, and return) must complete in well under 150 μ s per sensor on average. Vectoring plus memory-mapped access keeps each ISR to a handful of instructions, and non-critical processing (filtering, logging) is deferred to a lower-priority background/software task instead of being done inside the ISR.
- Latency control: Interrupt priorities are assigned in the VIC so higher-criticality sensors preempt lower ones, keeping worst-case latency bounded and predictable.

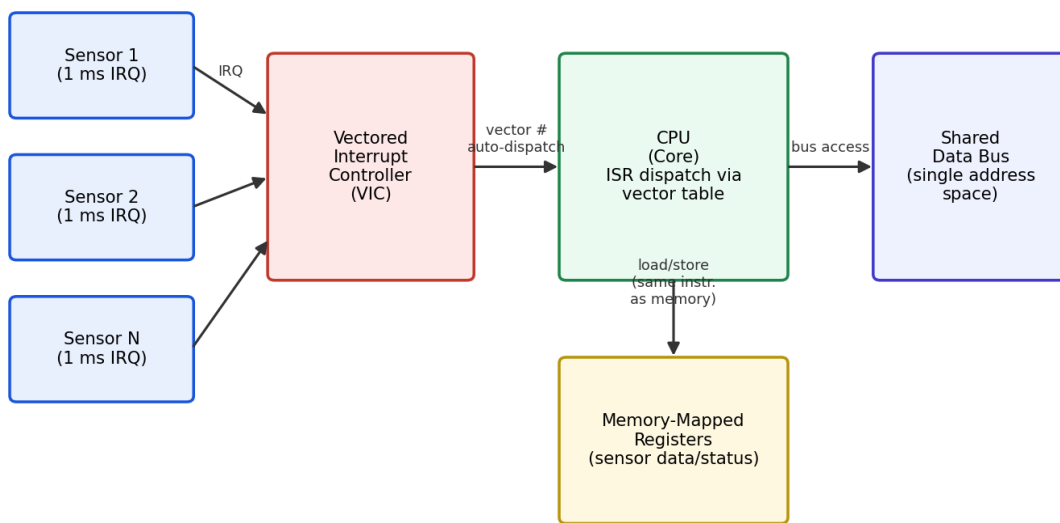


Figure 1: Vectored-interrupt, memory-mapped I/O path from sensors to CPU.

2. I/O Subsystem for High-Speed Storage (> 3 GB/s)

Constraints: throughput > 3 GB/s · must use NVMe over PCIe · CPU must not handle bulk data transfer · DMA controller with interrupt notification.

Design

- Interface choice: NVMe over PCIe is used because PCIe Gen3 x4 alone provides ~3.94 GB/s of raw bandwidth (Gen4 doubles it), comfortably clearing the 3 GB/s requirement, and NVMe's parallel command queues (up to 64K queues, 64K commands each) avoid the single-queue bottleneck of older SATA/AHCI.

- Offloading the CPU: The CPU never touches the data itself. It only writes a command (read/write, address, length) into an NVMe submission queue and rings a "doorbell" register. The NVMe controller then programs its own DMA engine to move the actual data.
- DMA with interrupt notification: The DMA engine performs the bulk transfer directly between the SSD and system RAM over the PCIe link. Once the transfer completes, the DMA engine raises a single completion interrupt (via MSI-X) to the CPU — one interrupt per completed I/O operation, not per byte or per word.
- Result: The CPU's load is limited to (a) issuing the command and (b) handling one lightweight completion interrupt, satisfying the "CPU should not handle bulk data transfer" constraint while sustaining > 3 GB/s throughput.

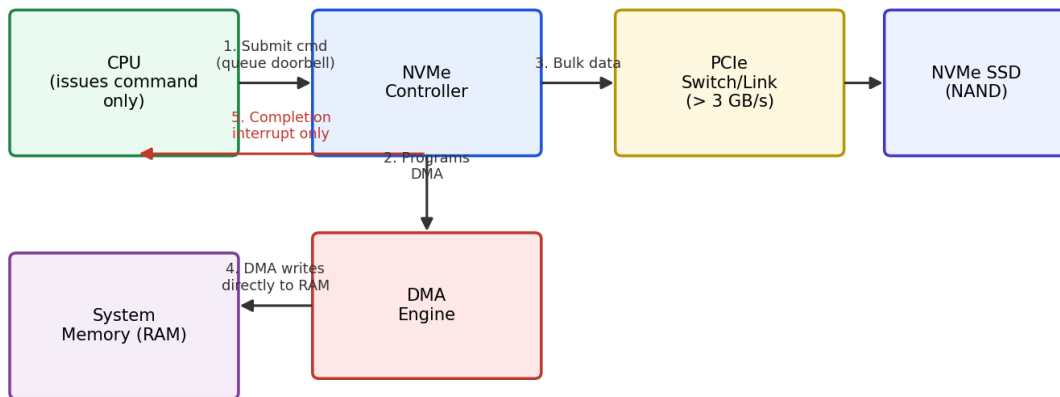


Figure 2: NVMe-over-PCIe data path with DMA offload and completion interrupt.

3. Multi-Device Communication Architecture (USB + Thunderbolt)

Constraints: 6 peripherals on USB, 2 high-speed devices on Thunderbolt · avoid interrupt starvation · use both hardware and software interrupts · ensure fair bus arbitration.

Design

- Bus separation: The 6 USB peripherals share the USB host controller's bus, while the 2 high-speed devices get dedicated Thunderbolt lanes (Thunderbolt is point-to-point/switched, not a shared bus like USB), so high-speed traffic does not compete directly with the USB devices for bandwidth.
- Fair arbitration: The USB host controller uses round-robin (frame-based, time-sliced) scheduling across its 6 devices so no single device can monopolize the bus. The Thunderbolt controller uses bandwidth-reservation/priority arbitration since its devices are latency- and bandwidth-sensitive.
- Avoiding interrupt starvation: A layered interrupt scheme is used — hardware interrupts (fixed, low-latency) are reserved for the high-priority Thunderbolt devices and for USB bus-level events, while lower-priority/background USB devices are serviced through software interrupts / threaded (deferred) ISRs. This ensures a burst of low-priority interrupts cannot block a high-priority device indefinitely, and vice-versa, because the interrupt arbiter enforces priority levels with aging/round-robin among equal-priority sources.
- Result: Every device gets guaranteed bus time and a bounded worst-case wait for interrupt service, satisfying both fairness and starvation-avoidance requirements simultaneously.

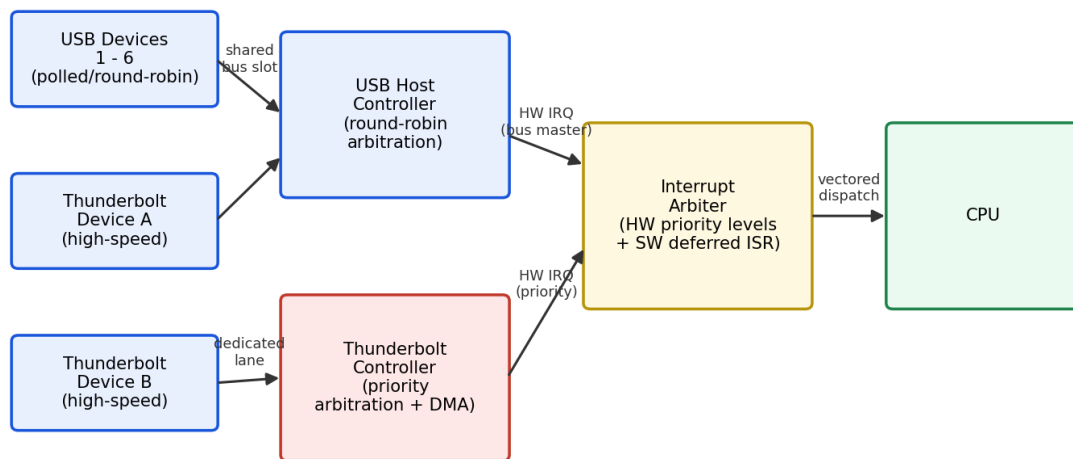


Figure 3: USB round-robin arbitration alongside Thunderbolt priority arbitration, unified by a layered interrupt arbiter.

4. Data Acquisition System (Continuous 500 MB/s Stream)

Constraints: continuous data stream at 500 MB/s · minimal CPU intervention · compare DMA vs interrupt-driven I/O · use memory-mapped I/O for device registers.

Design

- Device control: The acquisition device's control/status registers are memory-mapped, so the CPU can configure sample rate, start/stop, and buffer pointers using ordinary load/store instructions — no special I/O instructions needed.
- Data path: A DMA controller streams incoming samples directly into a ring buffer in RAM. The CPU is only interrupted after a full buffer (or half-buffer, for double-buffering) is filled, not for every sample.
- Why DMA over interrupt-driven I/O at this rate: at 500 MB/s, generating one interrupt per data unit would require millions of interrupts per second — the context-switch/ISR overhead alone would exceed available CPU cycles. DMA reduces this to a handful of interrupts per second (one per buffer), which is the only way to meet the "minimal CPU intervention" constraint.

Aspect	Interrupt-Driven I/O	DMA-Driven I/O
CPU involvement	CPU interrupted for every data unit transferred	CPU only sets up transfer and is notified once on completion
Throughput at 500 MB/s	Not feasible — interrupt overhead per byte/word saturates the CPU	Feasible — bulk block transfer directly between device and RAM
CPU utilization	Very high (context-switch overhead dominates)	Very low — CPU is free for other tasks
Best suited for	Low-frequency, low-volume events (keyboard, mouse)	High-frequency, high-volume continuous streams (this system)

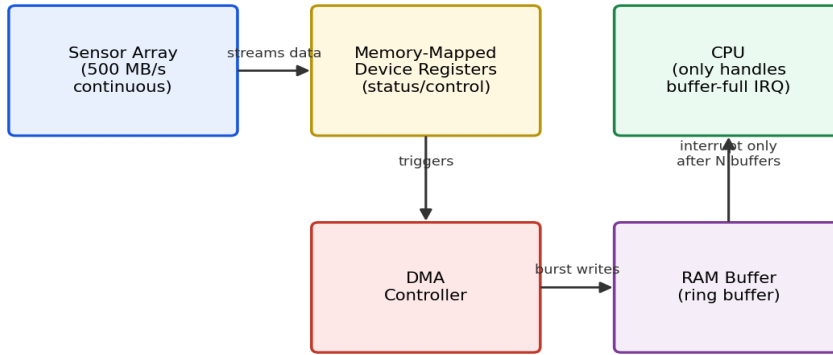


Figure 4: DMA-driven acquisition pipeline — CPU interrupted only on buffer completion.

5. Interrupt Handling Mechanism (Nested, $< 10 \mu\text{s}$ Latency)

Constraints: support nested interrupts · maximum interrupt latency $< 10 \mu\text{s}$ · vectored interrupts for high-priority devices · software interrupts for background tasks.

Design

- Priority-layered vector table: Interrupt sources are grouped into priority levels in a Nested Vectored Interrupt Controller (NVIC-style design). High-priority, time-critical devices use hardware vectored interrupts so the CPU jumps directly to their ISR address — no software dispatch loop, minimizing latency.
- Nesting: A higher-priority interrupt is allowed to preempt a currently executing lower-priority ISR. Each ISR entry pushes only the minimal context needed onto the stack, and priority levels are strictly enforced so a lower-priority interrupt can never delay a higher-priority one.
- Meeting $< 10 \mu\text{s}$ latency: Latency budget = vector fetch + context save + ISR dispatch. By keeping the vector table in fast on-chip memory, minimizing saved-register count, and avoiding software priority resolution for top-priority sources, the total worst-case path is kept under $10 \mu\text{s}$.
- Background tasks: Non-time-critical work (logging, statistics, UI updates) is triggered via software interrupts (trap/SWI instructions) at the lowest priority level, so it never competes with hardware-vectored, high-priority interrupts for the CPU's immediate attention — it simply runs whenever no higher-priority hardware interrupt is pending.

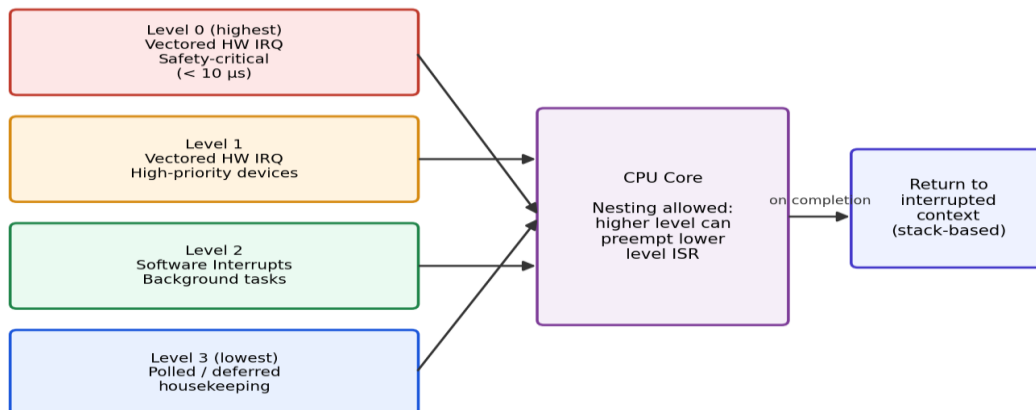


Figure 5: Nested priority levels — vectored hardware interrupts for critical devices, software interrupts for background work.